UNIVERSIDAD PANAMERICANA · CÓMPUTO DISTRIBUIDO

Adecuación de la calculadora a una arquitectura ADS

Proyecto del segundo parcial. Reporte técnico.

Antonio Annese Caballero
ID 0266972
20 de octubre de 2026

1. Objetivo

La calculadora del primer parcial resolvía suma, resta, multiplicación y división con una arquitectura de tres capas: un cliente gráfico, un middleware y servidores de cálculo. Este proyecto conserva esas cuatro operaciones y sustituye esa arquitectura por un sistema autónomo descentralizado. El acceso a cada servicio ya no pasa por un proceso que conozca a todos los demás. Pasa por un Campo de Datos que difunde mensajes, y por un Código de Contenido con el que cada célula decide, por su cuenta, si el mensaje le corresponde.

La configuración que se demuestra tiene cuatro nodos en malla completa y veinte células: cuatro interfaces gráficas y dieciséis servidores, cuatro de cada operación. El mismo programa de nodo y los mismos programas de célula sirven para otro número de instancias. Los identificadores, los puertos y la lista de vecinos se dan al arrancar; no están escritos como un tamaño fijo del sistema.

2. Qué cambió respecto al primer parcial

En el primer parcial el middleware era el punto por el que pasaba toda solicitud y todo resultado. Sabía qué clientes y qué servidores estaban registrados, y les reenviaba el mensaje. Si ese proceso se detenía, la calculadora dejaba de funcionar.

En esta versión no existe ese proceso. Un nodo del Campo de Datos no calcula, no elige qué servidor debe responder y no guarda un directorio global de células. Acepta conexiones de otros nodos y de las células que se le conectan, y retransmite cada mensaje de datos una sola vez. La célula servidora mira el Código de Contenido. Si es el de su operación, participa. Si es el de otra, lo ignora. La célula cliente publica la solicitud y, cuando ve una respuesta cuyo identificador de referencia es el de su solicitud, muestra el resultado.

3. Componentes

Hay tres programas, cada uno en su propio proceso.

Programa	Papel	Clase principal
Nodo del Campo de Datos	Forma la malla, acepta células y difunde mensajes.	`NodoApp`
Célula cliente	Captura los operandos, publica la solicitud y muestra la respuesta que le corresponde.	`ClienteApp`
Célula servidor	Se configura para una operación y, si le toca atenderla, calcula y publica la respuesta.	`ServidorApp`

La ventana del nodo muestra dos contadores: los mensajes que sí difundió y los duplicados que descartó para no dejarlos circular. La ventana del cliente muestra los dos operandos, el resultado y una bitácora. La ventana del servidor anuncia el servicio con un lema («YO SÉ SUMAR», «YO SÉ RESTAR», «YO SÉ MULTIPLICAR» o «YO SÉ DIVIDIR»), su huella y el código que escucha. La huella es un identificador de nueve dígitos, distinto en cada proceso, y viaja en los mensajes de esa célula.

4. Topología

Los cuatro nodos de la demostración escuchan en los puertos 7401, 7402, 7403 y 7404, todos en la misma máquina. Cada nodo abre una conexión TCP hacia los otros tres. La malla es completa: el camino entre dos nodos puede ser directo. Cada nodo, además, tiene conectadas cinco células: la interfaz, la suma, la resta, la multiplicación y la división.

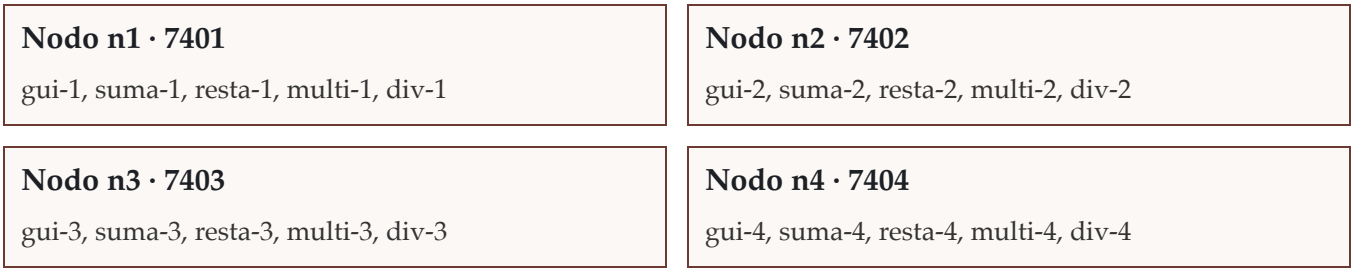


Figura 1. Cada nodo está unido a los otros tres. Las células solo hablan con el nodo al que se conectaron; entre ellas no hay canales directos.

Como los dos extremos de un par pueden intentar la conexión al mismo tiempo, a veces nacen dos TCP para el mismo vecino. El nodo se queda con una sola: la que inició el nodo de identificador menor. La otra se cierra. Los mensajes que hayan alcanzado a duplicarse en ese instante los corta el registro de identificadores, descrito más adelante. Si un enlace se cae, el nodo vuelve a intentar la conexión con ese vecino. No hace falta reiniciar el resto de la malla.

5. Protocolo de conexión

El saludo no se difunde. Solo vive en el TCP que se acaba de abrir. Una trama es una línea de texto UTF-8 con el prefijo `ADS1`.

Un nodo que llama a otro envía:

```
ADS1|HELLO|entidad=NODO|id=n1|puerto=7401
```

El nodo que acepta responde:

```
ADS1|HELLO_OK|id=n2|detalle=aceptado
```

Una célula envía:

```
ADS1|HELLO|entidad=CELULA|rol=SERVIDOR|id=suma-1|huella=902159599|servicio=SUMA
```

El rol de una interfaz es `GUI` y no trae servicio. El nodo contesta `HELLO_OK` con su propio identificador, anota la célula entre sus conexiones locales y, a partir de ahí, le entrega los mensajes del Campo de Datos. Cuando el socket de una célula se cierra, ese nodo publica un mensaje `WITHDRAW` para que el resto sepa que esa huella ya no está.

Nada de este saludo exige que existan exactamente cuatro nodos o cuatro células de cada tipo. Un quinto nodo se arranca con el mismo programa, otro puerto y una lista de vecinos. Una quinta célula de suma se arranca indicando el nodo al que quiere conectarse y el servicio `SUMA`.

---

6. Mensajes y Códigos de Contenido

Lo que circula por la malla es un mensaje de tipo `DATO`. El nodo no interpreta el código: lo copia hacia sus vecinos y hacia sus células, excepto hacia el enlace por el que acaba de llegar. La célula sí lo interpreta.

Campo	Uso
`id`	Identificador único de este mensaje. En una respuesta es nuevo; no se reutiliza el de la solicitud.
`ref`	En una respuesta o en un reclamo, el `id` de la solicitud original. En una solicitud va vacío.
`cc`	Código de Contenido.
`op1`, `op2`	Operandos, en una solicitud y copiados en la respuesta.
`res`	Resultado, solo en una respuesta correcta.
`estado`, `detalle`	`OK` o `ERROR`, y el motivo si la operación no se pudo hacer.
`origen`, `huella`	Célula que creó el mensaje y su huella.
`hops`	Saltos ya dados. El nodo lo incrementa al reenviar.

`ts`	Hora local de quien creó el mensaje.

Los valores se escapan para que un `|`, un `=` o un salto de línea no rompan la trama.

Operación	Solicitud	Respuesta	Reclamo
Suma	SUM_REQ	SUM_RES	SUM_CLAIM
Resta	SUB_REQ	SUB_RES	SUB_CLAIM
Multiplicación	MUL_REQ	MUL_RES	MUL_CLAIM
División	DIV_REQ	DIV_RES	DIV_CLAIM

`PRESENCE` y `WITHDRAW` no son operaciones. El primero lo publica una célula al conectarse, para anunciarse en el Campo de Datos. El segundo lo publica el nodo cuando esa célula se desconecta. Los reclamos (`*_CLAIM`) tampoco son el resultado: sirven para que varias células del mismo servicio se pongan de acuerdo sin un coordinador. La sección 8 explica por qué existen.

Una solicitud de suma se ve así:

```
ADS1|DATO|id=8f3a1c0e...|ref=|cc=SUM_REQ|origen=gui-
1|huella=228579469|op1=9|op2=4|res=|estado=|detalle=|hops=0|ts=16:50:12
```

La respuesta conserva la referencia y trae un identificador propio:

```
ADS1|DATO|id=b71e09aa...|ref=8f3a1c0e...|cc=SUM_RES|origen=suma-
2|huella=984499791|op1=9|op2=4|res=13|estado=OK|detalle=ok|hops=0|ts=16:50:12
```

---

7. Broadcast y prevención de ciclos

Una célula solo escribe en el nodo al que está conectada. Ese nodo entrega el mensaje a sus otras células y a sus nodos vecinos. Cada vecino hace lo mismo. En una malla completa el mismo mensaje regresa por otro camino: n1 se lo manda a n2 y a n3, y n2 también se lo manda a n3. Si nadie lo detiene, el mensaje no termina.

Cada nodo guarda los identificadores que ya procesó. Si un mensaje vuelve con el mismo `id`, el nodo no lo entrega otra vez y escribe en la bitácora «CICLO EVITADO». El contador ámbar de la ventana es exactamente ese número. Además, si el campo `hops` pasa de 16, el mensaje se tira. Dieciséis no es el tamaño de la red: es un tope por si el registro ya olvidó un identificador viejo y ese mensaje siguiera dando vueltas. El registro olvida identificadores con el tiempo para no crecer sin límite; en la demostración el corte ocurre por el identificador, mucho antes de esa depuración. Un mensaje sin `id` ni siquiera entra al registro: se rechaza.

La bitácora distingue vecinos de células. Si el mensaje llegó desde otro nodo, se reenvía a los vecinos restantes y no de vuelta al emisor. Por eso una línea puede decir «2 vecinos» aunque la malla tenga

tres enlaces. El estado «Malla completa · 3/3 pares» es el que indica que los tres enlaces están arriba. El «2» de un reenvío solo dice que uno de esos tres fue quien entregó el mensaje.

---

8. Una solicitud, varias células, una respuesta

En la demostración hay cuatro células de suma, una en cada nodo. El broadcast les entrega a las cuatro el mismo `SUM_REQ`. Si las cuatro calcularan y publicaran `SUM_RES`, el cliente vería cuatro resultados iguales y el Campo de Datos cargaría respuestas que no aportan nada. Hace falta una regla que elija una célula sin nombrar a un jefe.

La regla es local y usa solo lo que ya viaja en el Campo de Datos:

1. La célula recibe un `*_REQ` de su propio servicio. Si es de otro servicio, escribe que lo ignora y no hace nada más.
2. A los 60 ms publica un reclamo, `*_CLAIM`, con su huella. No calcula todavía.
3. Durante los siguientes 320 ms junta las huellas que se postularon a esa misma solicitud, incluida la suya.
4. Si su huella es la menor de ese conjunto, en orden de texto, calcula y publica la respuesta. Las huellas de la demostración tienen nueve dígitos, así que ese orden coincide con el numérico.
5. Si su huella no es la menor, no responde. Deja anotado a quién le tocó y programa un respaldo.
6. El respaldo de la siguiente huella espera 900 ms, y cada huella posterior suma 250 ms. Si en ese tiempo aparece la respuesta, el respaldo se cancela. Si no aparece, porque la célula ganadora se cayó después de postularse, la siguiente calcula y publica ella la respuesta.

Nadie pregunta a un registro central quién está vivo. La lista de postuladas es el conjunto de reclamos que de verdad llegaron. Una célula apagada no se postula, así que no puede ganar. Una célula que se postula y luego no responde deja el turno a la siguiente.

Hay una segunda barrera en el cliente. La respuesta trae en `ref` el identificador de la solicitud. El cliente solo pinta el resultado si ese `ref` es de una solicitud suya, y solo la primera vez. Otra `SUM_RES` con el mismo `ref` queda en la bitácora como duplicada y no cambia el número en pantalla. Así, aunque dos células llegaran a responder en una carrera muy justa, la interfaz no muestra dos resultados.

La división entre cero no es una excepción que tire el proceso. La célula que atiende publica `DIV_RES` con `estado=ERROR` y el detalle «División entre cero.» El cliente lo muestra como error.

---

9. Recorrido de una operación

1. En una célula cliente se escriben los operandos y se elige la operación.
2. La célula crea un `id` nuevo y un `DATO` con el Código de Contenido de solicitud, y se lo entrega a su nodo.
3. El nodo lo difunde a sus vecinos y al resto de sus células. Esos vecinos lo difunden a su vez.
4. Cada nodo que ya vio ese `id` lo descarta.

5. Las células servidor leen el código. Las de otra operación lo ignoran. Las de la operación pedida se postulan.
6. La de huella menor publica la respuesta, con `id` nuevo, `ref` de la solicitud, el resultado y el Código de Contenido de respuesta.
7. La respuesta también se difunde. No va dirigida a un cliente: cualquier interfaz la recibe.
8. Solo la interfaz que originó la solicitud la presenta. Las demás la ven en la bitácora como tráfico del Campo de Datos.

---

10. Descentralización

Ninguna función de las que el sistema necesita para seguir calculando vive en un solo proceso. La malla se arma entre pares: cada nodo conoce a los vecinos que le pasaron al arrancar y se conecta a ellos. El reenvío lo hace el nodo que tiene el mensaje, con su propia memoria de identificadores. La elección de quién responde la hacen las células comparando huellas, no un asignador. Si se detiene un servidor de suma, otra célula de suma atiende la siguiente solicitud. Si se detiene un nodo, las células que colgaban de él pierden el enlace, y los nodos restantes siguen difundiendo entre sí. El cliente de un nodo que sigue en pie puede sumar, restar, multiplicar y dividir.

Eso no significa que una célula funcione sin ningún nodo. Cada célula necesita el nodo al que se conectó, porque ese es su puerta al Campo de Datos. Lo que no existe es un nodo cuyo fallo apague a los demás.

---

11. Decisiones de implementación

El lenguaje es Java 11. El enunciado da puntos extra por usarlo, y el primer parcial ya estaba en Java, así que la bitácora, los sockets de una línea y la forma de las ventanas siguen siendo reconocibles. Cada ventana es un proceso aparte, con su propio log en `logs/`, para que en el video se vea que son programas distintos y no pestañas de uno solo.

El transporte es TCP. Una línea, un mensaje. TCP garantiza el orden dentro de un enlace, y eso importa: por el mismo socket primero viaja la solicitud y después los reclamos, así que una célula no decide antes de haber visto la solicitud. Entre enlaces distintos el orden no está garantizado; por eso la decisión espera 320 ms después del reclamo y no se toma en el instante en que llega el primero.

Se usaron códigos de reclamo además de los de solicitud y respuesta. Sin ellos, o todas las células de la operación responden, o hace falta un miembro que reparte turnos. El reclamo es un mensaje más del Campo de Datos: se difunde, se puede perder como cualquier otro, y el respaldo cubre el caso en que la ganadora no llegue a publicar el resultado. El cliente, de todas formas, se queda con la primera respuesta válida.

La aritmética está solo en la célula servidora. El nodo no suma. Una interfaz no suma. Si no hay ninguna célula del servicio pedido, el cliente espera cuatro segundos y escribe que no hubo respuesta, en lugar de calcularlo en local y disimular el fallo.

12. Qué se puede mostrar

`scripts/demo.sh` compila y abre las veinticuatro ventanas, acomodadas en cuatro columnas. `scripts/probar.sh` hace la misma malla sin ventanas y comprueba lo siguiente: la malla cierra los tres enlaces de cada nodo; 9+4 produce un solo `SUM_RES` con valor 13 y las otras tres células de suma ceden; repetir el mismo identificador no vuelve a entregar la solicitud; 8×7 lo atiende solo la multiplicación; 9−4 produce 5; 9÷0 regresa error; y, con una sola célula de suma viva, 20+3 produce 23.

El guion con el que se narró la demostración está en `Script_Presentacion.md`. El video está en https://drive.google.com/file/d/1B20m8tzX4MjSiu5_uAyjQYxr72L59dam/view?usp=sharing. El código fuente está en github.com/antonioannese/proyecto-parcial-2-computo-distribuido.

13. Conclusión

La calculadora sigue haciendo las cuatro operaciones, pero ya no depende de un middleware que conozca a todos. El Campo de Datos difunde, el Código de Contenido selecciona y el identificador del mensaje corta los ciclos de la malla. Cuando varias células saben hacer lo mismo, se postulan con su huella y responde la menor; si esa no contesta, responde la siguiente. Quitar una célula o un nodo saca de servicio solo a quien colgaba de ellos. El resto de la malla sigue calculando.